

Update Procedure

Last Updated: 9/5/2026

1.0 Updating Files

Files Needed:

- 1_CreateFolders.py
- 2_ProcessChar.py
- 3_VerifyPlugin.py

Unzip the “UpdateTools” zip folder.

Pre-Requisites:

Create a new folder “NewData_[GameVersion]”, and place the three above python files inside.

1.1 Creating Folder Structure

First, run the “1_CreateFolders.py” script to create the folder structure needed. This should create a bunch of folders called “Raw_[value]”.

1.2 Getting PCS Files

Need:

- Raw PCS data
- pcs.json

Go to Jedipedia, go to nodes section, find the “PCS” node. Right click, use the “Download JSON as ZIP” option. Save it to the “Raw_PCS” folder. Extract it so all the .json files are inside the “Raw_PCS” folder.

The str.pcs file is also needed. Navigate to Jedipedia, and use the following commands to save the “pcs.json” file to the base level folder (ie not in any of the raw folders):

```

function downloadJsonFile(data, filename) {

    const blob = new Blob([JSON.stringify(data, null, 4)], { type: 'application/json' });

    const url = URL.createObjectURL(blob);

    const a = document.createElement('a');

    a.href = url;

    a.download = filename || 'download.json';

    a.click();

    a.remove();

}

(async () => {

    const loaded = await stb.load('str.pcs');

    console.log('load result:', loaded);

    const ids = stb.ids('str.pcs');

    if (!ids) {

        console.log('No ids returned for "str.pcs" — the file name may be different now.');

        return;

    }

    console.log('Found', ids.length, 'ids');

    const out = {};

    for (const id of ids) {

        out[id] = stb.get(id);

    }

    downloadJsonFile(out, 'pcs.json');

})();

```

1.3 Getting Eye Color/Hair Color/ Skin Color Files

Need:

- Eyecolor JSON data
- Haircolor JSON data
- Skincolor JSON data

Navigate to the nodes section, find the dynamic node. For each of the “eyecolor”, “haircolor”, and “skincolor” node, right click, “Download JSON as ZIP”, and place them in the appropriate folders, and extract them.

1.4 Getting IPP/ITM Files

Need:

- IPP JSON data
- ITM JSON data

Navigate to the “IPP” and the “ITM” nodes in Jedipedia. For each, right click, and download them both as zipped JSONs and place them in the appropriate folder and extract them.

Note: since these are files with lots of items, if you right click and save them in Jedipedia, it may take a few minutes for the pop up box to appear.

1.5 Getting Dye Files

Need:

- ColorNameValues.json
- itmAppearanceColorsPrototype.json

Dyes need “ColorNameValues.json”, open up the browser, load Jedipedia, open up the console (ie press F12, go to console), and use the following code. Save it to the base level directory (ie not inside the raw folders).

```
function downloadJsonFile(data, filename) {

    const blob = new Blob([JSON.stringify(data, null, 4)], { type: 'application/json' });

    const url = URL.createObjectURL(blob);

    const a = document.createElement('a');

    a.href = url;

    a.download = filename || 'download.json';

    a.click();

    a.remove();

}

(async () => {

    const loaded = await stb.load('str.gui.colormames');

    console.log('load result:', loaded);

    const ids = stb.ids('str.gui.colormames');

    if (!ids) {

        console.log("No ids returned for 'str.gui.colormames' — the file name may be different now.");

        return;

    }

    console.log('Found', ids.length, 'ids');

    const out = {};

    for (const id of ids) {

        out[id] = stb.get(id);

    }

    downloadJsonFile(out, 'ColorNameValues.json');

})();
```

Dyes need “itmAppearanceColorsPrototype.json”. Go to Jedipedia, go to nodes, find the itmAppearanceColors” node, right click, “Download JSON”, and save it to the base level folder (ie not in any of the raw folders).

1.6 Get GOM Fields File

Need:

- GOM.fields.json

This file is used to translate various UI type names. Go to Jedipedia, open up the web console, and use this command, save it to the base level folder (not inside any of the raw folders):

```

(function() {

const text = Object.entries(GOM.fields).map(t => t.join("\t")).join("\n");

const blob = new Blob([text], { type: 'text/plain' });

const url = URL.createObjectURL(blob);

const a = document.createElement('a');

a.href = url;

a.download = 'GOM.fields.json';

document.body.appendChild(a);

a.click();

document.body.removeChild(a);

URL.revokeObjectURL(url);

})();

```

1.7 Getting ITM Name Values

Need:

- ITMNameValues.json

ITMNameValues.json is needed to translate ITM names from IDs/Values. Navigate to Jedipedia, and use the below command. Save the file to the base level folder (not in any of the raw folders).

```

function downloadJsonFile(data, filename) {

const blob = new Blob([JSON.stringify(data, null, 4)], { type: 'application/json' });

const url = URL.createObjectURL(blob);

const a = document.createElement('a');

a.href = url;

a.download = filename || 'download.json';

a.click();

a.remove();

}

(async () => {

const loaded = await stb.load('str.itm');

console.log('load result:', loaded);

const ids = stb.ids('str.itm');

if (!ids) {

console.log("No ids returned for 'str.itm' — the file name may be different now.");

return;

}

console.log('Found', ids.length, 'ids');

const out = {};

for (const id of ids) {

out[id] = stb.get(id);

}

downloadJsonFile(out, 'ITMNameValues.json');

})();

```

1.8 Getting Additional Language ITM Name Values

Need:

- ITMNameValues_French.stb
- ITMNameValues_German.stb

The French and German ITMNameValues.json is needed to be able to translate armor names to those languages. Go to Jedipedia, under the “Files” tab, navigate to “/resources/en-us/str/itm.stb”. You can see on the top right there is a Download Original option. You can also see you can open up the DE STB and the FR STB (at the top). You may need to have downloaded those assets via SWTOR (in the launcher, changing your language). For both the German and the French itm.stb file, download them to the “Raw_Langs” folder, and save them both as “ITMNameValues_German” and “ITMNameValues_French”.

1.9 Copy Blender Python Plugin File

Copy the “__init__.py” file into the base level folder (ie not in any of the raw files).

1.10 Run ProcessChar

Optional: Create a .txt file called “ResourceDirectory” with the path to the “Resources” directory that the face data will use (used for getting races). If this isn’t entered, this value can manually be entered once the 2_ProcessChar.py file is running.

Run “2_ProcessChar.py”. For example, in terminal, open it in this directory, and run “python 2_ProcessChar.py”. This will create a folder “jc_swtor_tools_[version]”.

Note this process can be sped up if the folder gets added to the exclusion list on Windows Defender. Don’t forget to delete the exclusion after.

1.11 JBA and Blend Files

Copy the .jba animation files and the .blend files into the “Anim” and “Blends” folder.

If you don’t do this, it will auto download them from this github page.

1.12 Verify Files

Run “3_VerifyPlugin.py”, this checks if there are missing files, sets some default values of the plugin if they are something different, and adds a “GAME_VERSION” to each function/class/variable” to make it unique. This then zips up the file and makes it ready for blender.

2.0 Portrait Export

Use tools in zip file: PortraitExport.zip

Pre-requisites: ImageMagick (<https://imagemagick.org/script/download.php>)

To cut down on time taken to try and match up existing characters in game to the blender plugin, instead of toggling through various options, one can take a look at the portrait tiles to quickly see and match up options from the game to the plugin. A toggle to cycle through all combinations of class/gender/species is also available.

- Enable the hidden/developer section of the plugin
- Go to the “Export Character Portrait Images (Batch)” section, set an output for the folder path.
- Cycle through each class/gender/species options
- Run “PortraitExport.bat”